



TP Raspberry Pi

C, C++, Python ...

Prof : **Kamal Boudjelaba**

17 septembre 2026

Table des matières

1	Introduction : l'environnement Linux du Raspberry Pi	3
1.1	Mettre à jour la liste des paquets	4
1.2	Supprimer un logiciel	4
2	Manipulation des fichiers et des répertoires	4
3	Écrire un programme avec Nano	5
4	Premier programme en C	6
4.1	Exécution	7
5	Premier programme en C++	7
5.1	Création du programme	7
6	Premier programme en Python	7
6.1	Création du programme	7
7	Les compilateurs et outils de développement	8
8	Thonny Python IDE	8
9	Geany	8
10	Architecture ARM et compilation	8
11	Exercice de synthèse	9
12	Écrire et exécuter des programmes sur le Raspberry Pi (Linux ARM)	10
12.1	Introduction : Éditeur de texte nano	10
12.2	Programme en C	12
12.3	Programme en C++	12
12.4	Programme en Python	12
12.5	Geany : éditeur de texte pour le développement sur Raspberry Pi	13
12.6	Thonny Python IDE	13

Préambule

Prérequis

- Connaissances de base de l'environnement Linux (commandes terminal).
- Notions élémentaires de programmation (structures de contrôle, variables).
- Utilisation d'un éditeur de texte en ligne de commande (comme **nano**).

Objectifs

- utiliser le terminal du Raspberry Pi sous Raspberry Pi OS ;
- manipuler des fichiers et des répertoires à l'aide des commandes Linux ;
- créer et modifier un fichier texte depuis le terminal ;
- compiler et exécuter un programme en C/C++ ;
- exécuter un programme en Python ;
- utiliser un environnement de développement graphique pour programmer en Python.

Compétences visées

- **C04** : Analyser un système informatique
- **C08** : Coder

Activités et tâches associées

- **D1 – Élaboration et appropriation d'un cahier des charges**
 - D1-T1 : Collecte des informations
 - D1-T2 : Analyse des informations
- **D2 – Développement et validation de solutions logicielles**
 - D2-T1 : Conception de l'architecture d'une solution logicielle
 - D2-T3 : Développement, utilisation ou adaptation de composants logiciels

1. Introduction : l'environnement Linux du Raspberry Pi

Le Raspberry Pi 5 est un ordinateur monocarte fonctionnant avec un processeur ARM 64 bits.

Dans ce TP, nous utiliserons **Raspberry Pi OS 64 bits**, le système d'exploitation officiel de Raspberry Pi. La version actuelle de Raspberry Pi OS est basée sur Debian GNU/Linux.

Le Raspberry Pi OS Desktop fournit un environnement graphique, mais il est également possible d'utiliser le système entièrement depuis un terminal.

Ouvrir un terminal depuis le menu du Raspberry Pi.

Les commandes présentées dans ce TP sont exécutées dans ce terminal.

Note : Le symbole \$ utilisé dans les exemples représente l'invite du terminal. Il ne doit pas être saisi. Certaines commandes nécessitent les droits administrateur et doivent être précédées de **sudo**.

Par exemple :

\$ pwd

signifie qu'il faut saisir :

pwd

Commande	Signification
pwd	Affiche le chemin du répertoire courant.
ls	Affiche le contenu du répertoire courant.
ls -l	Affiche le contenu du répertoire avec davantage d'informations.
cd répertoire	Change de répertoire.
mkdir répertoire	Crée un nouveau répertoire.
cp <source> <destination>	Copie un fichier ou un répertoire.
mv <source> <destination>	Déplace ou renomme un fichier ou un répertoire.
rm fichier	Supprime un fichier.
cat <fichier>	Affiche le contenu d'un fichier dans le terminal.
tree	Affiche l'arborescence d'un répertoire, si la commande est installée.
clear	Efface le contenu affiché dans le terminal.
man <commande>	Affiche la documentation d'une commande.

Le répertoire personnel

Chaque utilisateur possède un répertoire personnel.

Pour connaître le chemin de son répertoire personnel :

\$ echo HOME

Pour s'y rendre :

```
$ cd ~
```

ou simplement :

```
$ cd
```

Important : ne supposez pas que le répertoire personnel est `/home/pi`. Le nom de l'utilisateur est choisi lors de l'installation du système.

Pour connaître votre nom d'utilisateur :

```
$ whoami
```

Gestion des logiciels

Raspberry Pi OS utilise le système de gestion de paquets **APT**, hérité de Debian. APT permet notamment d'installer, de mettre à jour et de supprimer des logiciels.

1.1 Mettre à jour la liste des paquets

La commande suivante met à jour les informations concernant les paquets disponibles :

```
$ sudo apt update
```

Mettre à jour les logiciels installés

Pour mettre à jour les paquets installés :

```
$ sudo apt upgrade
```

Installer un logiciel

La syntaxe générale est :

```
$ sudo apt install <paquet>
```

Par exemple :

```
$ sudo apt install tree
```

1.2 Supprimer un logiciel

```
$ sudo apt remove <paquet>
```

Rechercher un paquet

```
$ apt search <mot-clé>
```

Par exemple :

```
$ apt search gcc
```

Remarque : les commandes `apt-get` et `apt` utilisent les mêmes mécanismes APT. Pour les commandes interactives utilisées dans ce TP, nous utiliserons `apt`.

2. Manipulation des fichiers et des répertoires

Dans la suite du TP, les fichiers seront placés dans le répertoire personnel de l'utilisateur.

Créer un répertoire de travail

Créer un répertoire nommé Prog :

```
$ cd  
$ mkdir Prog  
$ cd Prog
```

Vérifier le contenu du répertoire :

```
$ ls
```

Créer une arborescence

Créer les répertoires suivants :

```
$ mkdir C  
$ mkdir CPP  
$ mkdir Python
```

Vérifier l'arborescence :

```
$ tree
```

Si la commande **tree** n'est pas disponible :

```
$ sudo apt install tree
```

Copier et déplacer un fichier

La commande **cp** permet de copier un fichier :

```
$ cp fichier.txt destination
```

La commande **mv** permet de déplacer ou renommer un fichier :

```
$ mv fichier.txt nouveau_nom.txt
```

Supprimer un fichier

```
$ rm fichier.txt
```

Attention : la commande **rm** supprime directement les fichiers. Ils ne sont pas placés dans la corbeille.

3. Écrire un programme avec Nano

Présentation

Nano est un éditeur de texte fonctionnant dans le terminal.

Vérifier qu'il est disponible :

```
$ nano --version
```

Si nécessaire, il peut être installé avec :

```
$ sudo apt install nano
```

Créer un fichier

Pour créer ou modifier un fichier :

```
$ nano nom_du_fichier
```

Par exemple :

```
$ nano test.txt
```

Il n'est normalement pas nécessaire d'utiliser **sudo** pour modifier les fichiers appartenant à votre utilisateur.

Principaux raccourcis de Nano

Le symbole ^ affiché par Nano représente la touche **CTRL**.

Raccourci	Fonction
CTRL + O	Enregistrer le fichier.
CTRL + X	Quitter Nano.
CTRL + W	Rechercher du texte.
CTRL + K	Couper la ligne courante.
CTRL + U	Recoller une ligne coupée.
CTRL + G	Afficher l'aide.
CTRL + A	Aller au début de la ligne.
CTRL + E	Aller à la fin de la ligne.

4. Premier programme en C

Création du programme

Se placer dans le répertoire destiné aux programmes C :

```
$ cd ~/Prog/C
```

Créer le fichier :

```
$ nano ProgC.c
```

Saisir le programme suivant :

```
#include <stdio.h>

int main(void)
{
    printf("Le programme en C a ete execute.\n");
    return 0;
}
```

Enregistrer le fichier puis quitter Nano.

Compilation

Un programme écrit en C doit être compilé avant d'être exécuté.

Utiliser le compilateur GCC :

```
$ gcc -Wall -Wextra -o ProgC ProgC.c
```

Les options **-Wall** et **-Wextra** demandent au compilateur d'afficher davantage d'avertissements utiles pendant le développement.

Après compilation, vérifier la présence de l'exécutable :

```
$ ls
```

4.1 Exécution

Exécuter le programme :

```
$ ./ProgC
```

Observer le résultat affiché dans le terminal.

5. Premier programme en C++

5.1 Création du programme

Se placer dans le répertoire C++ :

```
$ cd ~/Prog/CPP
```

Créer le fichier :

```
$ nano ProgCPP.cpp
```

Saisir :

```
#include <iostream>

int main()
{
    std::cout << "Le programme en C++ a été exécuté." << std::endl;
    return 0;
}
```

Enregistrer puis quitter Nano.

Compilation

Compiler le programme avec g++ :

```
$ g++ -Wall -Wextra -o ProgCPP ProgCPP.cpp
```

Exécution

```
$ ./ProgCPP
```

6. Premier programme en Python

6.1 Création du programme

Se placer dans le répertoire Python :

```
$ cd ~/Prog/Python
```

Créer le fichier :

```
$ nano ProgPython.py
```

Saisir :

```
#!/usr/bin/env python3

print("Le programme en Python a été exécuté.")
```

Enregistrer puis quitter Nano.

Exécution

Contrairement au C et au C++, aucune compilation préalable n'est nécessaire pour ce programme Python. Exécuter le programme avec Python 3 :

```
$ python3 ProgPython.py
```

Vérifier que le message apparaît dans le terminal.

Remarque : Raspberry Pi OS fournit Python 3. Il est préférable d'utiliser explicitement la commande `python3` afin de ne pas dépendre d'un éventuel alias `python`.

7. Les compilateurs et outils de développement

Vérifier la présence de GCC

Le compilateur C utilisé précédemment est GCC.

Vérifier sa présence :

```
$ gcc --version
```

Vérifier également le compilateur C++ :

```
$ g++ --version
```

Si ces commandes ne sont pas disponibles, installer les outils de compilation :

```
$ sudo apt update
```

```
$ sudo apt install build-essential
```

Le paquet **build-essential** fournit notamment les outils nécessaires à la compilation de programmes C et C++.

8. Thonny Python IDE

Présentation

Thonny est un environnement de développement intégré (IDE) destiné principalement à Python.

Il est particulièrement adapté à la programmation car il propose notamment :

- un éditeur de code avec coloration syntaxique ;
- l'exécution directe des programmes ;
- une console Python ;
- des outils de débogage ;
- une interface adaptée aux débutants.

Thonny est fourni avec l'édition graphique standard de Raspberry Pi OS.

Exécuter le programme Python avec Thonny

Lancer Thonny depuis le menu des applications.

Ouvrir le fichier :

ProgPython.py

puis exécuter le programme.

Comparer le résultat obtenu avec celui obtenu précédemment depuis le terminal.

Question : quelles sont les différences entre l'exécution d'un programme Python depuis le terminal et son exécution depuis Thonny ?

9. Geany

Geany est un éditeur de texte léger orienté développement.

Il peut être installé si nécessaire avec :

```
$ sudo apt install geany
```

Geany permet notamment de travailler avec des fichiers C, C++, Python et de nombreux autres langages.

Remarque : contrairement à Thonny, qui est fourni avec l'édition graphique standard de Raspberry Pi OS, Geany ne doit pas être considéré comme une application nécessairement préinstallée.

10. Architecture ARM et compilation

Le Raspberry Pi 5 utilise un processeur ARM 64 bits.

Les programmes C et C++ compilés directement sur le Raspberry Pi sont donc compilés pour l'architecture du Raspberry Pi.

Pour observer l'architecture du système :

```
$ uname -m
```

Sur un Raspberry Pi OS 64 bits, le résultat attendu est généralement :

```
aarch64
```

Pour obtenir davantage d'informations :

```
$ uname -a
```

11. Exercice de synthèse

Créer dans le répertoire /Prog l'organisation suivante :

```
Prog/  
  C/  
    ProgC.c  
    ProgC  
  CPP/  
    ProgCPP.cpp  
    ProgCPP  
  Python/  
    ProgPython.py
```

Travail demandé

1. Vérifier l'architecture du système avec `uname -m`.
2. Vérifier la présence de GCC et G++.
3. Compiler le programme C avec :

```
gcc -Wall -Wextra -o ProgC ProgC.c
```

4. Exécuter le programme C.
5. Compiler le programme C++ avec :

```
g++ -Wall -Wextra -o ProgCPP ProgCPP.cpp
```

6. Exécuter le programme C++.
7. Exécuter le programme Python avec :

```
python3 ProgPython.py
```

8. Exécuter ensuite le programme Python avec Thonny.
9. Modifier les trois programmes afin qu'ils affichent votre nom et votre classe.

Conclusion

- Le Raspberry Pi 5 est un véritable ordinateur fonctionnant sous Linux.
- Raspberry Pi OS est basé sur Debian et utilise APT pour la gestion des logiciels.
- Le répertoire personnel dépend du nom de l'utilisateur créé lors de l'installation.
- En C et en C++, le programme doit être compilé avant son exécution.
- En Python, le programme peut être exécuté directement avec Python 3.
- Sur un Raspberry Pi 5 équipé de Raspberry Pi OS 64 bits, l'architecture du système est généralement `aarch64`.
- Les commandes et procédures évoluent : la documentation officielle reste la référence à consulter lorsqu'une commande ou un outil change.

12. Écrire et exécuter des programmes sur le Raspberry Pi (Linux ARM)

12.1 Introduction : Éditeur de texte nano

Nano est un éditeur de texte simple mais puissant, conçu pour les systèmes Unix et Linux. Il permet de créer ou modifier facilement des fichiers texte.

Certains systèmes (comme macOS et Linux) incluent généralement Nano par défaut. Pour vérifier s'il est installé :

```
nano --version
```

Pour l'installer sur Debian ou Ubuntu :

```
sudo apt-get install nano
```

— Démarrer Nano sans ouvrir de fichier spécifique :

```
sudo nano
```

— Créer ou modifier un fichier existant :

```
sudo nano Nom_Fichier.Extension
```

Exemple :

```
sudo nano prog1.c
```

— Si le fichier n'est pas dans le répertoire courant, préciser le chemin complet :

```
sudo nano /home/pi/MesProgrammes/prog2.py
```

Si le fichier spécifié n'existe pas, Nano le créera automatiquement. Si aucun nom n'est indiqué, un fichier temporaire sera créé, et Nano demandera un nom lors de la fermeture.

Au bas de la fenêtre, Nano affiche des raccourcis clavier. Le symbole $\wedge$ indique une combinaison avec la touche **CTRL** (ex. : $\wedge O = \text{CTRL} + O$).

Pour enregistrer : **CTRL** + O Pour quitter : **CTRL** + X Si des modifications ont été faites, Nano demandera si vous souhaitez les enregistrer : appuyez sur Y (oui), ou N (non), puis Entrée.

Raccourci	Fonction
CTRL + A	Aller au début de la ligne
CTRL + E	Aller à la fin de la ligne
CTRL + Y	Faire défiler vers le haut
CTRL + V	Faire défiler vers le bas
CTRL + G	Ouvrir l'aide de Nano
CTRL + O	Sauvegarder le fichier. Permet de modifier ou confirmer le nom avant enregistrement
CTRL + W	Rechercher une expression. Pour relancer la recherche, utiliser ALT + W
CTRL + K	Couper la ligne courante
CTRL + U	Coller une ligne précédemment coupée
CTRL + X	Quitter Nano (demande de sauvegarde si nécessaire)

12.2 Programme en C

1. Créer un dossier nommé ProgNano dans /home/pi
2. Créer un fichier nommé ProgC.c :

```
sudo nano /home/pi/ProgNano/ProgC.c
```

3. Écrire le programme suivant, enregistrer puis quitter Nano :

```
#include <stdio.h>

int main(void)
{
    printf("Le programme en C nommé ProgC.c et écrit dans nano a été exécuté.\n");
    return 0;
}
```

4. Compiler le programme :

```
gcc -o TestC ProgC.c
```

5. Exécuter le programme :

```
./TestC
```

12.3 Programme en C++

1. Créer un fichier nommé ProgCPP.cpp dans /home/pi/ProgNano :

```
sudo nano /home/pi/ProgNano/ProgCPP.cpp
```

2. Écrire le programme suivant, enregistrer puis quitter Nano :

```
#include <iostream>
using namespace std;

int main(void)
{
    cout << "Le programme en C++ nommé ProgCPP.cpp et écrit dans nano a été exécuté." << endl;
    return 0;
}
```

3. Compiler le programme :

```
g++ -o TestCPP ProgCPP.cpp
```

4. Exécuter le programme :

```
./TestCPP
```

12.4 Programme en Python

1. Créer un fichier nommé ProgPython.py dans /home/pi/ProgNano :

```
sudo nano /home/pi/ProgNano/ProgPython.py
```

2. Écrire le programme suivant, enregistrer puis quitter Nano :

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
print("Le programme en Python nommé ProgPython.py et écrit dans nano a été exécuté.")
```

3. Exécuter le programme (pas besoin de compilation) :

```
python ProgPython.py
```

12.5 Geany : éditeur de texte pour le développement sur Raspberry Pi

Geany est un éditeur de texte orienté développement, disponible sur Windows, macOS, Linux, et sur les Raspberry Pi équipés de processeurs ARM. Il offre une interface graphique agréable, avec notamment :

- la coloration syntaxique ;
- la prise en charge de nombreux langages (C, C++, Python, HTML, PHP, Ruby, Java...) ;
- un accès rapide à la compilation et à l'exécution.

Geany est préinstallé avec Raspberry Pi OS Desktop. Il se trouve dans le menu **Programmation**, avec d'autres outils comme Thonny Python IDE.

Exécuter du code depuis Geany

- Enregistrer le script
- Construire (Build)
- Compiler : les résultats s'affichent dans l'onglet **Compilateur** en bas de la fenêtre
- Exécuter : un terminal s'ouvre automatiquement et lance le programme

12.6 Thonny Python IDE

Thonny est un IDE (environnement de développement intégré) minimaliste conçu pour Python. Il est particulièrement adapté aux débutants. Thonny intègre son propre interpréteur Python, ce qui permet d'exécuter des scripts sans configuration préalable.